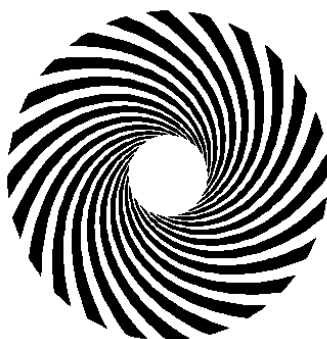


Bureau of Meteorology



ERTS Data Formats

July 2003

Version 1.0 draft 1

ERTS Data Formats, version 1.0 draft 1

Date of Issue: July 2003

Copyright: © 2003 by Commonwealth Bureau of Meteorology

All rights reserved. No part of this document may be reproduced by any means or in any form without the express permission of the Commonwealth Bureau of Meteorology.

This documentation was prepared for Commonwealth Bureau of Meteorology by:

Bureau of Meteorology
Hydrology Section
Flood Warning Program Office

Table of Contents

Background.....	5
Technical Discussion.....	6
Terminology	6
Transmitted Information	7
ALERT Binary Format (ABF).....	8
BCC Extended Check Binary Data Format.....	8
<i>Problems with Binary ALERT Format.....</i>	<i>9</i>
Enhanced ALERT Binary Data Format (EAF).....	9
Enhanced IFLOWS (EIF) Format.....	10
<i>Differences between EIF and EAF</i>	<i>10</i>
<i>EIF Implementation Issues</i>	<i>11</i>
<i>Possible Problems with the EIF</i>	<i>11</i>
Appendix A ALERT Specification	13
Data Message Formats.....	13
Binary Format.....	13
Enhanced Binary Format	14

List of Figures and Tables

Table 1 : Data in Binary Representation within a Message	7
Table 2 : Sensor Identification in Binary Representation within a Message	7
Table 3 : Binary Data Format	8
Table 4 : BCC Extended Check Binary Data Format.....	8
Table 5 : Enhanced ALERT Binary Data Format.....	9
Table 6 : Enhanced IFLOWS Format	10
Table 7 : Message Format Identifier	13
Figure 1 : Message Format – An Example	14





Background

Various data transmission formats have been associated with the Event-reporting Radio Telemetry Systems (ERTS) since the beginning of the use of the ALERT (Automatic Local Evaluation in Real Time) concept for local flash flooding systems.

The Bureau of Meteorology has been using the **Binary Format** in all its installations and the Local Agency systems it has directly associated itself so far. It is the only recommended data transmission format specified in the Bureau of Meteorology Equipment Specification A788, first issued in July 1988 and revised in March 1993. (See Appendix A)

Since the implementation of the first ALERT system various developments have taken place. Among them, it is worth noting the development of the **BCC Check Signal** (Brisbane City Council) which has been implemented in the present hardware as an option. This format was implemented to be transparent to the Binary Format so that the QNX2 ALERT software originally ignored it. Since then, it has been incorporated into the ALERT software and is now in use by other systems as well.

A new development for Australia is the interest in the Enhanced IFLOWS Format (EIF), which has been widely implemented in the East Coast of USA in the IFLOWS systems. This format has some error checking capabilities through the use of CRC values and has attracted some Australian users here.



Technical Discussion

This section describes some of the technical details of the format used in the transmission of ERTS data.

Terminology

The following are some of the descriptions of terminology used.

Frame

This is the window used by the decoder to convert bit signals to binary data. Indirectly it is the minimum size of a complete message that would be meaningful on its own right.

Word

A collection of message bits that form a message frame. Words are separated by IDLE state. They are also referred to as **bytes**

Address

The address of a message is the description of the originator. In ERTS formats, the ERTS ID provided to each sensor at its site forms the address.

Event data

The event data is the information the message contains in its body.

Header

The header of a word or a message is used to identify the start of a word or a message.

Asynchronous transmission

The receiver of an asynchronous transmission has to re-lock itself to the start of each message in order to decode the message. Conversely, in synchronous transmission, both the sender and the receiver are working to a common timer.

Simplex transmission

A simplex transmission is one way from a permanent sender to a permanent receiver. The sender has no way of knowing how successful it has been in sending the transmission intact.



Transmitted Information

The transmission format is expected to transmit the sensor identity number which uniquely identifies each sensor and the corresponding data value. It has to be noted that in ERTS, the time associated with the data is obtained from the base station computer which time stamps the data, and hence is not required to be transmitted with the data. So, essentially the transmission format is very simple for ERTS. As it is also a simplex transmission it is not necessary to wrap other information around the data packet that is usually needed for full duplex transmission. These are related to information passed back to the sender to confirm reception of data.

The ERTS data is stored as an integer within the electronic equipment and sent as an integer between 0-2047 range.

The following is an example of how real data is converted to a binary form for transmission. The data value is 2316 from a sensor with an sensor ID of 142, will be transmitted as a set of the binary data given in Table 1.

Element Number	0	1	2	3	4	5	6	7	8	9	10
Data Value	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇	D ₈	D ₉	D ₁₀
Value of Element	0	1	1	1	0	0	0	1	0	0	0

Table 1 : Data in Binary Representation within a Message

This example shows that the data is represented as 11 bits and the value of 2316 is equal to a binary representation of 01110001000. The maximum value possible is 2047 when it will be 1111111111. The minimum value of 0 is equal to 00000000000.

Element Number	0	1	2	3	4	5	6	7	8	9	10	11	12
Sensor ID	A ₀	A ₁	A ₂	A ₃	A ₄	A ₅	A ₆	A ₇	A ₈	A ₉	A ₁₀	A ₁₁	A ₁₂
Value of Element	0	0	1	1	0	0	0	0	1	0	0	1	0

Table 2 : Sensor Identification in Binary Representation within a Message



This example shows that the sensor ID is represented as 13 bits and the value of 142 is equal to a binary representation of 0011000010010. The maximum value possible is 8191 when it will be equal to 111111111111. The minimum value of 0 is equal to 0000000000000.

The above shown data and the sensor ID have to be now represented with the necessary header to form the data transmission packet.

ALERT Binary Format (ABF)

This is one of the early formats developed for ALERT systems by the US National Weather Service and is still in widespread use. The first two bytes have start bits of 01 and the last two bytes have start bits of 11. Of the total available bit numbers of 32 from the four 8-bit bytes, eight bits are used in this way, leaving 24 bits for data and sensor ID. Of the 24 bits, 13 are used for sensor IDs and 11 for data.

The start bits are shown on the right as it is convention to show transmission from right to left.

Stop	0	1	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Start
Stop	0	1	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	Start
Stop	1	1	D ₄	D ₃	D ₂	D ₁	D ₀	A ₁₂	Start
Stop	1	1	D ₁₀	D ₉	D ₈	D ₇	D ₆	D ₅	Start

Table 3 : Binary Data Format

BCC Extended Check Binary Data Format

The BCC check signal format is the same as the binary format for the transmission of the event data. For the check signals that are transmitted at set intervals to signal the health of the equipment, a second 4 byte message is sent immediately after sending a binary format check signal for each sensor defined at the site in the field.

Stop	0	1	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Start
Stop	0	1	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	Start
Stop	1	0	HD ₄	HD ₃	HD ₂	HD ₁	HD ₀	A ₁₂	Start
Stop	1	0	DE	VCO	BS ₃	BS ₂	BS ₁	BS ₀	Start

Table 4 : BCC Extended Check Binary Data Format



In the electronic equipment, the data is stored as a 16 bit value, and only the lower 11 bits are sent with every message because of the limitation of the message format. In this check signal, the top five bits are sent. The software makes use of this information to re-assemble the full data which is stored as a 16 bit integer in the database. The series of HD values are the high data bits.

This format also enables a battery check status to be transmitted. BS are the battery status bits. An error detection bit VCO is also used, but this is not implemented in all hardware. Another error status bit DE (Data error), possibly an analogue sensor out of range is also used.

So, the above two formats are the ones that are used extensively in Australia and accepted by the current QNX2 ALERT software used by the Bureau of Meteorology and Local Agencies in co-operative flood warning systems.

Problems with Binary ALERT Format

The above formats suffered from the following disadvantages. They became more apparent as more complex and denser networks were developed:

- Lost reports. As the data transmission from several sites are independent, data can be lost when they occur around the same time. Usually, the weaker signal will be discarded.
- Corruption of data. Data collisions, noise or interference or poor radio path can corrupt the message.

To overcome these problems, other variations of the binary format have been tried out.

Enhanced ALERT Binary Data Format (EAF)

Previously in USA, an enhanced ALERT format has been tried out with some error detection capabilities. The need for error detection arose from the following commonly observed problems:

Stop	1	1	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Start
Stop	D ₁	D ₀	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	Start
Stop	D ₉	D ₈	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	Start
Stop	C ₀	C ₁	C ₂	C ₃	C ₄	C ₅	B	D ₁₀	Start

Table 5 : Enhanced ALERT Binary Data Format

This format limits sensor IDs to 12 bits giving a range of 0-4096 only. The data is still 11 bits. However, the start bits for all except the first byte is now used. This could result in higher incidence of data lost through collision. This format was abandoned in the end.



The B is a battery status bit, and the C bits are the CRC check bits. The CRC check bits C's are also used to transmit wind gust data for wind sensors. No CRC check is possible for wind sensors. The receiving software has to determine first whether the sensor is wind sensor or not before doing the CRC checks using the C bit stream.

Enhanced IFLOWS (EIF) Format

The origins of the concept of the **Integrated Flood Observing and Warning System (IFLOWS)** goes back to the National Flash Flood Program Development Plan in 1978. The US National Weather Service (NWS) began a joint effort with selected states in the Appalachian Region of the United States to undertake the establishment and development of a flash flood warning system to improve flood warning capabilities in that Region.

Stop	1	1	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	Start
Stop	D ₀	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	Start
Stop	D ₈	D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	Start
Stop	R ₀	R ₁	R ₂	R ₃	R ₄	R ₅	D ₁₀	D ₉	Start

Table 6 : Enhanced IFLOWS Format

where:

A_i is the ith bit of the sensor address;

D_i is the ith bit of the event data;

R_i is the ith bit of the frame check sequence (FCS), ie. check values

EIF uses the two high-order bits of the first byte to indicate the message format. Since no valid ABF or EAF message has 11 in these bits, all formats remain uniquely identifiable. Like ABF, EIF provides a 13-bit address and an 11-bit data field, so any ABF message can be converted to EIF without data loss.

Differences between EIF and EAF

The principal reason for creation of EAF and EIF was to incorporate a CRC, so that some error detection capability can be introduced. However, EIF and EAF differ in two areas, battery status and wind gust/CRC substitution. EAF provides a "battery status" bit by shortening the gage identifier field from 13 to 12 bits, consequently reducing the number of unique identifiers from 8192 to 4096. EIF has no such battery status bit. Instead, it is suggested that battery condition should be treated like any other measurement; i.e. the sensor should be assigned an identifier and report battery voltage, rather than a simple good/bad indication.

The second difference between EAF and EIF relates to wind gusts. EAF allows wind sensors to substitute gust information for the CRC bits. EIF does not permit this, and as wind data transfer is now treated differently anyway, is no longer an issue.



EIF Implementation Issues

The base station receiving software will support any mix of ABF or EIF gauges. As EIF uses the same signaling, frequencies, 4-byte message format, 13-bit address, and 11-bit event data as ABF, very little changes to hardware should be necessary. EIF simply rearranges the fields within a message and adds an easily calculated FCS, so only minor firmware changes should be required for its implementation.

Base station software developers should also be able to add EIF support readily. When the software has determined that a message does not follow ABF specifications, the message should be tested for EIF (two high-order bits of first byte equal to 11 and a valid FCS). If the message passes, the address and event data fields are extracted and processed as if the message were ABF.

To provide maximum vendor flexibility while ensuring compatibility among various devices, some of the following guidelines could be followed:

- We could have the gauge firmware support both ABF and EIF and the selection of the required format is done at the time the gauge is installed.
- EIF firmware must be available as a standalone product so previously purchased gauges can be upgraded easily. Ideally, such a product would be a field-replaceable (socketed) ROM/PROM chip.
- EIF-aware repeaters must recognize and decode EIF messages, and discard (without repeating) any EIF message with an invalid FCS.
- EIF-aware repeaters could be made to convert both ABF and EIF format messages to EIF before retransmission.
- EIF aware decoders could be made to check EIF format messages and convert them to ABF to pass on to the collection software, while letting through ABF messages without any CRC checking.

In all cases, the need for the employment of EIF should be verified thoroughly prior to any decision being taken to convert over.

Possible Problems with the EIF

As the data transmission is asynchronous, and the messages do not have a distinctive frame, maintaining a timer is required. The base station software, decoder or the repeater will start a timer upon receipt of the first byte of a (potential) message. If three more bytes are not received before the timer expires, the decoder discards what was received and awaits the first byte of the new message. Since the timer cannot be started until the first byte arrives, the minimum duration possible is 100 ms based on the time needed to receive 3 bytes (ie 30 bits) at 300 baud rate. However, due to the apparent coarseness of PC-DOS timers, with a resolution of about 55 ms, the frame window averages about 275 ms. This increased window is likely to increase the reception of random noise. And as the EIF depends on the first byte only as the frame start identifier, it will liable to discard more messages. In contrast, the ABF can uniquely identify between the first two bytes and the last two bytes. Therefore it can work a timer half as long.



For EIF, additional checking of parameters such as the ratio of invalid messages need to be generated by the decoding software to monitor situations where large scale rejection of messages may take place due to a long duration noise. Long duration noise is quite feasible to be caused by an open receiver squelch circuit. This information should be monitored by the user to make sure that good messages are also not being discarded.

Effects of Random Noise on Gauge Formats

All formats encode 24-bits of intelligence in a 32-bit envelope. All 16,777,216 data bit combinations are valid, but for each, the remaining 8 bits must form a singular pattern or the message is marked invalid. Thus in the presence of pure random noise, one spurious message in 256 ($=2^{*8}$) is possible. This is approximately about 0.4% regardless of the message format.

However, suppose a stronger second signal arrives at the start of the second byte of a previously initiated message. As no checking is done at the second byte level, this message will be construed as forming the second byte and so on of the previous message. While this message will be subsequently discarded through the CRC checking, the next result is two messages being lost. However, for ABF, as the checking is done for each byte, at least the second message has a chance to be accepted

Field Experience on Gauge Formats

It is important to collect as much as possible data on field trials for the different formats.



Appendix A ALERT Specification

This is an extraction from the Equipment Specification A788 (Revision 2)
Provision of an Event-Reporting Radio Telemetry System

Data Message Formats

The data message is made up of four (4) bytes (words), each ten (10) bits long. This includes the start and stop bits. Four possible message formats have been defined with each format having a unique two bit identifier in the first byte of the message. The defined formats are:

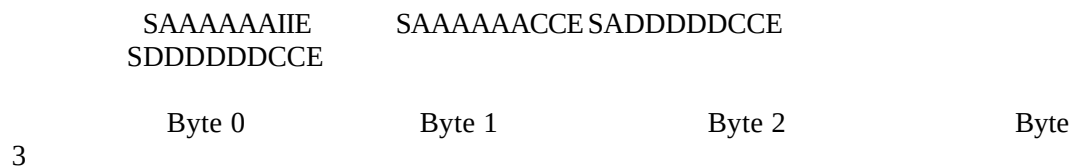
2 Bit Identifier	Message Format
01	ASCII
10	BINARY
11	ENHANCED BINARY (with CRC Checks)
00	Extended Messages (future 1200 baud use)

Table 7 : Message Format Identifier

Of these, only the binary format has been used so far in Australia.

Binary Format

This format uses negative logic and is used by the current field hardware.



Bit uses are:

S = four start bits

A = thirteen sensor identification bits

I = one pair of format identifier bits (=10 for binary format)

C = three pairs of check bits (=10 for byte 1 and 11 for bytes 2 and 3)

D = eleven data bits



E = four stop bits

The message is transmitted as 4 ten bit (including start and stop bits) bytes (words) using negative logic. Each bit grouping such as the sensor identifier or the data is transmitted from least to most significant bit. An example for a station identifier number of 2316 and a data accumulator number of 142 is shown with the order of the transmitted bits within the groups as indicated below:

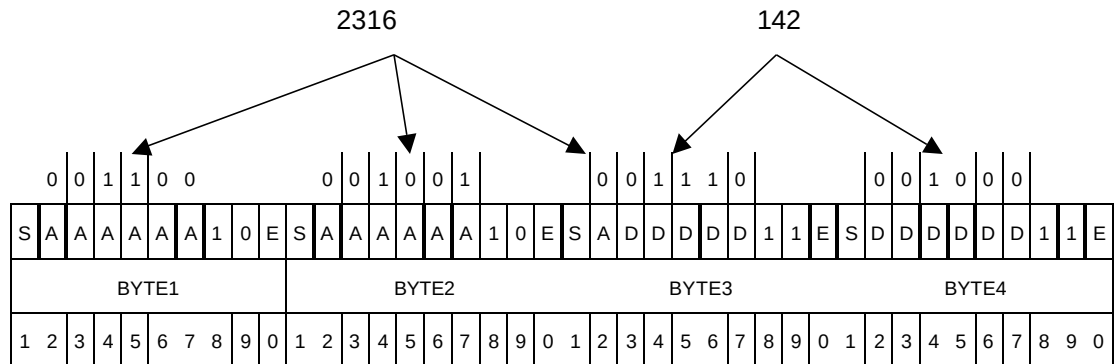


Figure 1 : Message Format – An Example

Enhanced Binary Format

Another format with internal error detection capabilities has been considered in the United States. This is referred to as 'enhanced binary format'. This format however has not been implemented due to operational problems. Prospective suppliers should note the existence of this format, however there are no plans for any further implementation in Australia at this stage